KAZAKH-BRITISH TECHNICAL UNIVERSITY

Object-Oriented Programming & Design
Final Project

10 points

Instructor: Izbassar Assylzhan
Referenced from: Pakizar Shamoi

Deadline: $14 - 15^{th}$ **week, on your practice time. No late submissions will be accepted! Please, read carefully class requirements. Remember about naming conventions. Keep your classes separately.**

The project requires much more effort & time, rather than practices. For that reason, please, plan your time carefully! Good luck! :)

Quick remainder

For the 2nd attestation, we have following point distribution:

1. Practice works - 4 pts.
2. Lab 3 - 5 pts.
3. Endterm - 10 pts.
4. Project (Part A: Use Case & Class Diagrams) - 5 pts.
5. Project (Part B: Implementation) - 5 pts.

Introduction

The current project is research-oriented university system. You should have classes (superclasses, subclasses, abstract classes), interfaces, enumerations, your own exceptions, patterns (to be studied next week), etc. - all techniques we have studied. Before coding you will need to design your system - create an architecture using **Use Case** and **UML Class Diagrams**.

Your project costs **30 points**: 10 pts in 2*nd* attestation, and 20 points put your final exam. **Note that**, other 20 points of the final exam will be oral defense, and team leaders will automatically passed for this part.

There will be 3 parts for this project.

Part A: Diagrams

There have to be Use Case & Class diagrams. The defense will be on 14th week.

You can use StarUML, TopCoder, GenMyModel, or any other tool you like. But, you have to remember that the only requirement is to be able to convert your class diagram to code. Better to use of these options, they are both free and have everything we need. For a topcoder installation, some Mac users will need to disable security settings, instructions are here, or here. Upon finishing, generate code sources from your class diagram and fill the classes with methods implementations. **Don't forget** about reverse engineering - reflecting back changes to your diagrams.

Part B: Models (Classes)

Show draft of your classes during 15th week (practice time), final version on final exam. For this part you are not obliged to simulate the system at work, you will just need to create models - classes, interfaces, enums, etc.

Part C: Demo (Run your system in the console)

So, for your final exam, you should finish & correct your previous parts, then SUBMIT YOUR REPORT(.pdf only) and PROJECT (.zip only, with documentation) and PRESENTATION (.pdf) in our MS Teams channel (we will create needed folders).

The **report** must be complete, detailed and well-structured. Include the detailed description of your classes, interfaces, etc. Include code fragments and final versions of UML diagrams in your report. In addition, provide info about problems you had and project management issues, screenshots of the teams or telegram group.

The **presentation** should have no more than 3-4 slides (what works/what does not work).

Requirements

We have some general requirements for this project. So, you should use and hold the next requirements:

- Object-oriented style: *Low coupling, high cohesion,* usage of Comparable, Comparators, equals, hashcode, toString, etc. Usage of java api (standard classes);
- Consistency with UML and intuitive usage;
- Any user should access the system via authentication;
- Properly working serialization (think about Data Storage and some pattern);
- Do not forget about proper usage of enumerations. You can use them to represent teachers' position, for example - TUTOR, LECTOR, SENIOR_LECTOR, PROFESSOR, etc.;

- Proper and logically consistent usage of Collections;
- Documentation;

There won't be detailed description of the project, because of it is not only a programming task, but also a *DESIGN* task, so it is up to you which fields & methods will be in your classes. Even though, its required that we will be able to have following classes: **`User`**, **`Employee`**, **`Teacher`**, **`Manager`**, **`Student`**, **`GraduateStudent`** (can be Master or PhD student), **`Admin`**, **`Course`**, **`Mark`**, **`Lesson`**, **`TechSupportSpecialist`**, **`Researcher`**, **`ResearchPaper`**, **`ResearchProject`**, **`News`**, **`Message`**. **Note:** You can also add other classes.

System specifications

MOST IMPORTANT FUNCTIONALITY - course registration, putting marks, research. Try to finish them first. In general, your system have to support next processes & types:

- Lesson types: LECTURE/PRACTICE;
- Switching between languages: KZ, EN, RU;
- The class of general & graduated students;
- A teacher must have a method to send a complaint about student(s) to a dean with urgency levels as LOW, MEDIUM, HIGH;
- Major, minor, free elective courses. Please note, that for a SITE student some major course from "Oil and Gas school" can be a free elective.
- News with comments. News with a topic "Research" must be prioritized in order (pinned). When some Researcher publishes a paper, there must be an announcement. Also, don't forget to automatically generate news about top cited Researcher in the university.
- The `Researcher` class must have a method to calculate h-index;
- All graduated students have their research supervisor, who is a `Researcher`. If a person whose $h - index < 3$ is assigned as a supervisor, a custom exception must be thrown.
- There can be more than one instructor per course, i.e. for lecture & practice times separately;
- In the system, teachers & students **CAN** be researchers. Those teachers, who are Professors, PhD & Master students **ARE** always researchers. However, bachelor students and other teachers (e.g., tutors, senior lectures, etc.), can also be researchers. Moreover, there can be some employees, who is neither a teacher nor student, but he is a researcher. Researcher has a research projects(s), research papers (also an Object!) and etc.;
- The fields for a `ResearchPaper` can be chosen from LMS Logs and Student Performance: The Influence of Retaking a Course. You can take 5-10 important ones: citations, name, authors, journal, pages, date, doi etc.;
- Researcher must have a method `printPapers(Comparator c)` that prints his research papers in sorted order, dictated by the comparator - by date published or by citations or by the article length (use pages). Moveover, the system must support printing research papers of

all researcher in the university, also sorted by date published or by citations or by the article length. Additionally, there should be method that support printing top cited researcher of the school, of the year (among all schools);

- The `ResearchPaper` must have a method `String getCitation(Format f)`, where format can be either "Plain Text" or "Bibtex". To see the text of formats, you can follow the link above, and click "Cite this" button. Then, you can find both format texts. The `ResearchProject` has a topic, published papers, and project participant. If someone who is not a `Researcher` tries to join into the `ResearchProject`, there should be thrown custom `Exception`;
- Report generation (about marks, just simple statistics). Additionally, there should be "Working official messages", about some events inside the university (for example, like booking the room when the exam is planned and so on);
- The tech support specialists need to be able to see new requests, and they **CAN** accept/reject them. After getting and seeing the request, its status should be VIEWED, and other statuses can be ACCEPTED, REJECTED, DONE (e.g. of request: to fix a projector or printer);
- The graduated students must have a list of published research paper(s) as diploma projects;
- As it is a research university, it has its own university researh journals. All users in the university (not only researchers) can subscribe to some university journals. The system must notify readers when the new paper in published in the journal they are subscribed. From time to time, new journals appear. Which pattern is this?
- Use 4 or more design patterns.

General checklist

1. `Admin`
   - Manage Users (add, remove, update);
   - See log files about user actions;
2. `Teacher`
   - View courses;
   - Manage Course;
   - Put marks;
   - View students, info about students;
   - Send messages to other employees (actually, any employee can send the message to any employee);
   - Send complaints.
3. `Student`
   - View Courses, Register for Courses;

- View info about teacher of a specific course;
- View Marks;
- View Transcript;
- Rate teachers;
- Get Transcript;
- Student organizations. Student can be a member/head.

4. `Manager`

   - Assign courses to teachers;
   - Approve students registration, Add courses for registration (specify for which major/year of study the course is intended);
   - Manager types – OR, Deparments, etc. (use enumeration);
   - Create statistical reports on academic performance;
   - Manage news;
   - View info about students and teachers (in different ways , e.g., sorted by gpa , alphabetically , etc);
   - View requests from employees (they have to be signed by dean/rector).

5. `Researcher`

   - You should think about `Researcher` class. Is it an interface? Abstract class? Created using Decorator pattern? Just employee? Figure it out. There is no single answer.

Important notes:

- Students can't have more than 21 credits;
- Students can't fail more than 3 times;
- Mark consists of 1st, 2nd attestation, and final.

Bonus: Take into account as many details as possible (for valuable extra features you will get extra points, e.g. Schedule generation (take into account room load, room type, etc.), Attendance, Report generation option for teacher (about marks), advanced search by regular expressions, startups, recommendation letters.